PROJECT-BASED LEARNING

PHASE-II EVALUATION

VidOptix -VideoCompressionSystem

Team ID: DSCPP-III-2026-T198

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team Information

Team ID: DSCPP-III-2026-T198
Semester: 3rd
Section: B,DS1,ML1,ML2
Domain: *Data Structure and C++*

Team Members

1. *Anshika Goswami – 2510380062*
2. *Yaduveer Yadav – 2510370438*
3. *Deepanshi Choudhary – 2510370456*
4. *Mohd Ubaid – 2510370084*

Mentor

Mr. Kartikey Arora

Interactions completed: 4

BitSquad is developing VidOptix in the Data Structures and C++ domain, our responsibilities include algorithm development, video processing, enhancement, interface design, and testing.

Phase-I Recap & Feedback Incorporated

Phase-I Status

The project was proposed as a frame-based video compression system intended to reduce video file size while keeping acceptable visual quality.

Feedback Received

- *Improve quality preservation alongside compression.*
- *Complete the compression pipeline.*
- *Make the interface easier to use.*

Changes Incorporated

- *Enhancement methods added to project scope.*
- *DCT / quantization / coding integration planned.*
- *UI workflow for settings and output preview planned.*

Brief Discussion: Phase-II work focuses on incorporating quality preservation, optional enhancement, and a more complete workflow.

Problem Statement & Finalized Objectives

Problem Statement

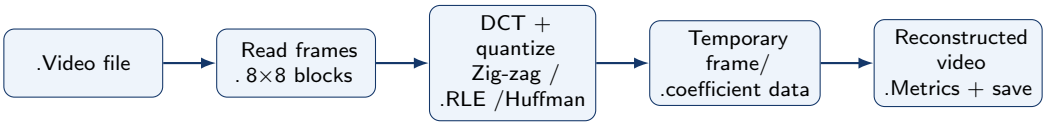
Video files require substantial storage. VidOptix aims to reduce video size through frame-based compression while maintaining acceptable visual quality and exploring enhancement techniques.

Finalized Objectives

1. *Read video frames and divide them into 8×8 blocks.*
2. *Apply DCT and integrate quantization, zig-zag scan, RLE and Huffman coding.*
3. *Provide optional enhancement and denoising.*
4. *Evaluate file-size reduction, processing time and visual quality.*

Brief Discussion: The finalized scope balances compression efficiency, visual quality and usability.

Updated System Architecture / Workflow



Major Components

- *Frame reader and extractor*
- *8×8 block DCT and compression stages*
- *Optional enhancement / denoising*

Changes Since Phase-I

- *Quality preservation is an explicit focus.*
- *Enhancement and denoising options planned.*
- *UI preview and performance comparison emphasized.*

Brief Discussion: The pipeline is local and does not require network communication for video processing.

Module Design & Functional Requirements

Major Modules

1. *Video input and frame extraction*
2. *8×8 block processing and DCT*
3. *Quantization and coefficient coding*
4. *Enhancement / denoising*
5. *Output reconstruction and evaluation*

Functional Requirements

- *Accept a video file from the user.*
- *Offer low / medium / high compression options.*
- *Allow optional enhancement and denoising.*
- *Preview and save the output video.*

Module Dependency

Input → frame reader → DCT/compression → optional enhancement → output writer → quality/size evaluation

Brief Discussion: Modules are designed to connect into a single application; full end-to-end integration remains in progress.

Technology Stack & Development Methodology

Technology Stack

- C++: *Core implementation and object-oriented program structure.*
- OpenCV: *Video input/output, frame extraction, and image processing.*
- CMake : *C++ project configuration and build management.*
- VS Code: *Development and debugging environment.*
- Github: *Source-code version control and team collaboration.*

Development Methodology

1. Finalize compression, enhancement, interface, and evaluation requirements.
2. Implement and refine the selected compression and enhancement techniques.
3. Develop video processing, compression, enhancement, and output components.
4. Connect the processing modules with the user interface and file-handling workflow.
5. Test different videos and compare file size, visual quality, and processing time.

Brief Discussion: C++ supports algorithm implementation; OpenCV handles video frames, while encoding support may be required for practical video output..

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures in C	*Queue, Priority Queue (Min Heap), Binary Tree, Vectors*	*Frame Processing, Huffman Coding, Data Storage*
OOPs with C++	*Classes, Objects, Encapsulation, Constructors, File Handling*	*Video, Frame, Enhancer, Compressor, HuffmanCoder Classes*

Development & Implementation Progress

Module	Status	Work Completed
Frame Reading & Extraction	Completed	*Video input handling and frame extraction*
DCT & 8×8 Block Division	Completed	*Frame division into blocks and DCT processing*
Compression Pipeline	In Progress	*Quantization, zig-zag scanning, RLE and Huffman coding*
Enhancement & Integration	Pending	*Enhancement implementation, UI integration and testing*

Overall Project Completion: *75%*

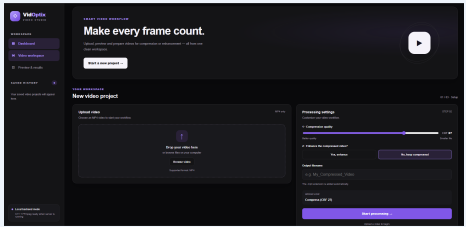
Brief Discussion: Our project is approximately 75% with major initial stages such as frame extraction and DCT-based processing developed. We are currently working on the remaining compression stages, enhancement, and complete system integration.

Working Prototype / Demonstration

Implemented Features

- *Video input and frame extraction*
- *8×8 block division and DCT processing*
- *Compression pipeline development using quantization, zig-zag scanning, RLE and Huffman coding*
- *User-interface development for compression options and enhancement*

Prototype / Output



Brief Discussion:We are developing VidOptix as a user-friendly video compression system with different compression options and enhancement features. Our goal is to let users process videos, compare the output, and save the compressed video while maintaining acceptable visual quality.

Testing, Results & Validation

Test / Module	Expected Result	Observed Result	Status
Video Input & Frame Extraction	*Input video is loaded and frames are extracted*	*Frame-processing module implemented*	Pass
Frontend–Backend Integration	*UI communicates with backend processing modules*	*Frontend UI connected to backend*	Pass
Video Compression & Output	*Compressed video is generated with reduced file size*	*File-size and quality evaluation required*	*Completed*

Results / Validation

Performance metrics to evaluate
1.Video file-size reduction
2.Compression processing time
3.Output video quality
4.Successful video processing and output saving

GitHub Repository & Development Activity

Repository Information

- GitHub: `https://github.com/yaduveery41/pblvidoptix3rdsemesterpbl_VidOptix_3rd_semester`
- README.md: *Updated with project information.*
- Contributors: *All four team members added.*
- Repository Status: *Latest project updates uploaded.*
- Branches: *Phase II*

Development Activity

- *Regular commits maintained*
- *Meaningful commit messages*
- *Individual contribution visible*
- *Latest implementation uploaded*
- *Documentation maintained*

Individual Contribution

Member	Role	Technical Contribution	GitHub Commits
Yaduveer Yadav	*Core Development*	*Video input, frame reading and extraction; DCT and 8×8 block division*	*XX*
Anshika Goswami	*Team Lead / UI*	*Frame extraction support and user-interface development*	*XX*
Mohd Ubaid	*UI / Enhancement*	*User-interface development and enhancement module collaboration*	*XX*
Deepanshi Choudhary	*Compression / Enhancement*	*DCT and 8×8 block division collaboration; enhancement module development*	*XX*

Brief Discussion:Our team has divided the work among video processing, compression algorithms, enhancement, and user-interface development. Each member is contributing to specific modules, and we are coordinating our work to integrate everything into one functional system.

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- *Balancing video compression ratio with acceptable visual quality.*
- *Combining DCT, quantization, zig-zag scanning, RLE, and Huffman coding into one pipeline.*
- *Implementing Huffman coding using min-heaps, priority queues, and binary trees while managing frame-processing efficiency.*

Solutions / Learning

- *Dividing video frames into 8×8 blocks and developing the compression pipeline in stages.*
- *Understanding DCT, quantization, run-length encoding, Huffman coding, and video enhancement techniques.*

Phase-III Roadmap

1. *Finish the compression pipeline and enhancement features.*
2. *Connect the compression, enhancement, and user-interface modules.*
3. *Measure file-size reduction, processing time, and output video quality.*
4. *Validate the system using different input videos and compression settings.*
5. *Prepare the final report, implementation details, and performance evaluation.*

Brief Discussion: The main challenges involve preserving video quality while reducing file size and integrating multiple algorithms. Phase III will focus on completing the remaining modules, integrating the system, testing performance, and validating the final prototype.

Expected Outcomes & References

Expected Final Outcomes

- *A fully functional video compression system.*
- *Measurable compression and quality improvements.*
- *Useful for real-world video processing.*
- *Potential to handle larger workloads.*
- *Scope for future research and publications.*

References

1. *DCT-based image/video compression research.*
2. *Huffman coding and lossless compression.*
3. *Data Structures and Algorithms.*
4. *OpenCV and FFmpeg documentation.*

Thank You
Questions & Suggestions

Brief Discussion: VidOptix aims to reduce video file size while preserving visual quality, with enhancement features and performance evaluation planned as part of the remaining development.